جـامعة الحسن الثـاني بالدار البيضاء
UNIVERSITÉ HASSAN II DE CASABLANCA

ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS
UNIVERSITÉ HASSAN II DE CASABLANCA

C# Lab

A turn-based duel in the console · C# for Unity · Master's in Immersive Technologies, Digital Engineering for Health Professions

Abdelhakim EL Ghayoubi

ENSAM Casablanca · 2026–2027

Contents

Before you start 1
1 Getting started 3
2 The duel 6
3 Going further 18
4 Reference 23

Before you start

One session of about two hours, in the console: no Unity, no 3D scene. A console program reads and writes text in a terminal, which makes every step visible. You start with four short exercises, then you write a small game: the player, Aya, fights three enemies, one at a time, turn by turn. The exercises of part 3 are done at home.

Table 1: The four parts of the lab.

Part	You write	Main ideas
1	Four small programs: a character sheet, a health status, a poison, a heal	Variables, `if`, `for` loop, method, reading an error
2	The duel: a player, three enemies, a game loop	Classes, properties, inheritance, `List<T>`, `while`, safe input
3	A boss, an event, several games, a results file	`virtual`/`override`, `event`, LINQ, `try`/`catch`
4	Nothing: this is the reference (commands, errors, frequent questions)	

Everything you write here carries over to Unity: a `Player` class with a `Health` property, a list of enemies, a `Died` event. Only the display changes. In Unity, `Console.WriteLine` becomes `Debug.Log`.

What you need

- **The .NET 10 SDK**, installed from Microsoft's website (lecture notes, section 0.8).
- **A code editor**: Visual Studio Code with the *C# Dev Kit* extension, or Visual Studio.
- **A terminal**: the one in VS Code (menu **Terminal > New Terminal**), or Windows Terminal.
- **The lecture notes**, Part 1: each step names the section that explains the idea.

Check the installation

Open a terminal and type `dotnet --version`. A version starting with 10 should appear. If the terminal answers that `dotnet` is not recognized, see part 4.



Figure 1: `dotnet --version` prints the SDK version; `dotnet --list-sdks` lists the installed SDKs.

Where to keep the projects

Create a short folder, for example `D:\CSharpLab`, and open the terminal there (`cd D:\CSharpLab`). Each project is a subfolder: `WarmUp` for part 1, `Arena` for the duel. Avoid long paths inside OneDrive: some tools fail there.

How to read a step

Steps are numbered and end with a *checkpoint*: what you should see when the step works. Do not move to the next step before you reach the checkpoint: a mistake is much easier to find in ten new lines than in a hundred.

Every output in this document is a real program output, and the screenshots come from a real terminal. The code of parts 2 and 3 gives the structure and leaves `// TODO` lines: writing those lines is your job.

> **Watch out**
>
> The fights use random numbers. So that the outputs in this document can be repeated, they were produced with `new Random(7)`: a fixed *seed* always gives the same sequence of numbers. With

> `new Random()`, your numbers will be different in every game, and that is expected.

1 Getting started

This part checks that everything works and brings back the basics of Part 1 of the course: create a project, run it, write a few lines, read an error. Each exercise fits in a single `Program.cs` file.

Step 1: create and run a project

1. In the terminal, go to your working folder, then type `dotnet new console -n FirstProgram`. The command creates the folder `FirstProgram` with two files: `FirstProgram.csproj` (the description of the project) and `Program.cs` (the code).
2. Enter the folder (`cd FirstProgram`) and type `dotnet run`. The SDK compiles the project, then runs it.
3. Open the folder in VS Code: `code .` (the dot means the current folder).

Checkpoint: the terminal prints `Hello, World!`.



Figure 2: The three commands and their result. The first `dotnet run` takes a few seconds: it compiles before it runs.

For each exercise below, you can either create a new project (`dotnet new console -n Exercise1`) or replace the content of `Program.cs` in the same project. Keep a copy of what you replace.

> **Watch out**
>
> The .NET 10 project template adds `using System;` by itself. In this lab, every file still writes it, as in Unity, where you must write it.

Exercise 1: a character sheet

Declare five variables for Aya: her name (`string`), her level (`int`, 3), her health (`int`, 100), her speed (`float`, 5.5) and a Boolean that says whether she has a key (`bool`, false). Print them as below. The last line is computed: the next level needs `level × 100` experience points.



Figure 3: Checkpoint of exercise 1.

Hints. Section 1.2: a `float` literal is written with an `f` (`5.5f`). An interpolated string, `$"Health: {health} HP"`, replaces each `{...}` with its value. The spaces after `Name:` line up the values.

> **Watch out**
>
> On a Windows set to French, your program prints `Speed: 5,5 m/s`, with a comma. The display of a number follows the language of the system; the code itself is always written with a dot. The course projects set a neutral display, hence the dot in the screenshot.

Exercise 2: the health status

With a variable `health`, print a message that depends on Aya's health:

Table 2: The four states.

Health	Message
more than 70	`healthy`
31 to 70	`injured`
1 to 30	`critical`
0	`Aya is down`

```
Checkpoint of exercise 2, with health = 45
```

```
Aya has 45 HP: injured
```

Then change the value of `health` (100, 12, 0) and run again: each value must give its message.

Hints. Section 1.4: an `if` / `else if` / `else` chain. The order of the tests matters: C# stops at the first true test, so start with the highest threshold.

Exercise 3: the poison

Aya has 40 health points and she is poisoned: each turn, she loses 8 points. Write a `for` loop over six turns that prints the health left. Health never goes below 0, and when it reaches 0, the program prints `Aya is down.` and leaves the loop.



Figure 4: Checkpoint of exercise 3: the loop stops at turn 5, before the sixth.

Hints. Section 1.5: `for (int turn = 1; turn <= 6; turn++)`. `break` leaves the loop at once. **Question:** with a poison of 5 instead of 8, what does the program do? Predict the answer, then check it.

Exercise 4: a healing method

Write a method `Heal` that receives the current health, the amount of healing and the maximum health, and returns the new health, never above the maximum. Call it three times:

Checkpoint of exercise 4

```
50 + 30 -> 80
90 + 30 -> 100
0 + 25  -> 25
```

Hints. Section 1.6: the signature is `int Heal(int health, int amount, int maxHealth)`, and `Math.Min(a, b)` returns the smaller of the two numbers. In a file with top-level statements, the method can be written after the lines that call it.

Step 2: read a compiler error

The compiler refuses a program that breaks the rules of the language, and it says where and why. Delete the `;` at the end of line 4 of exercise 4 (the first `Console.WriteLine`), then type `dotnet build`. *Checkpoint:* the build fails with this message.

```
Program.cs(4,53): error CS1002: ; expected
```

The message reads from left to right:

1. **The file and the position**: `Program.cs(4,53)` means line 4, column 53, just after the parenthesis.
2. **The code**: `CS1002`. A search for "CS1002" on Microsoft Learn gives a page that explains it.
3. **The sentence**: what the compiler expected.

The same slip does not always give the same message. In the screenshot below, the `;` is missing after `int health = 100`. A declaration can go on with a comma (`int a = 1, b = 2;`), so the compiler announces an expected comma (`CS1003`) at the end of line 3. When a message looks absurd, look at what is missing at the position given, then at the end of the line before.

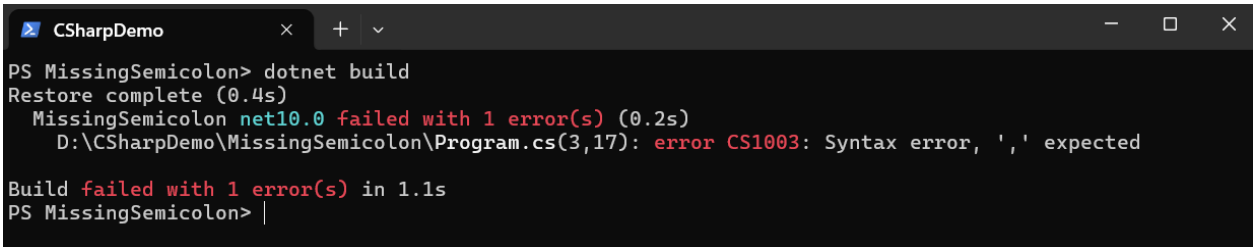


Figure 5: Another missing semicolon, in another program: the message speaks of a comma.

Put the `;` back, then try another error: delete the last `}` of the file. The message becomes `Program.cs(10,49): error CS1513: } expected`. Part 4 lists the errors you are most likely to meet in this lab (section 1.15 of the notes).

2 The duel

The player, Aya, fights a wave of three enemies, one at a time: a Goblin, an Orc, then a second Goblin. Each turn, she attacks or drinks a potion, then the enemy strikes back. The game ends when all enemies are defeated, when Aya has no health left, or when she runs away.

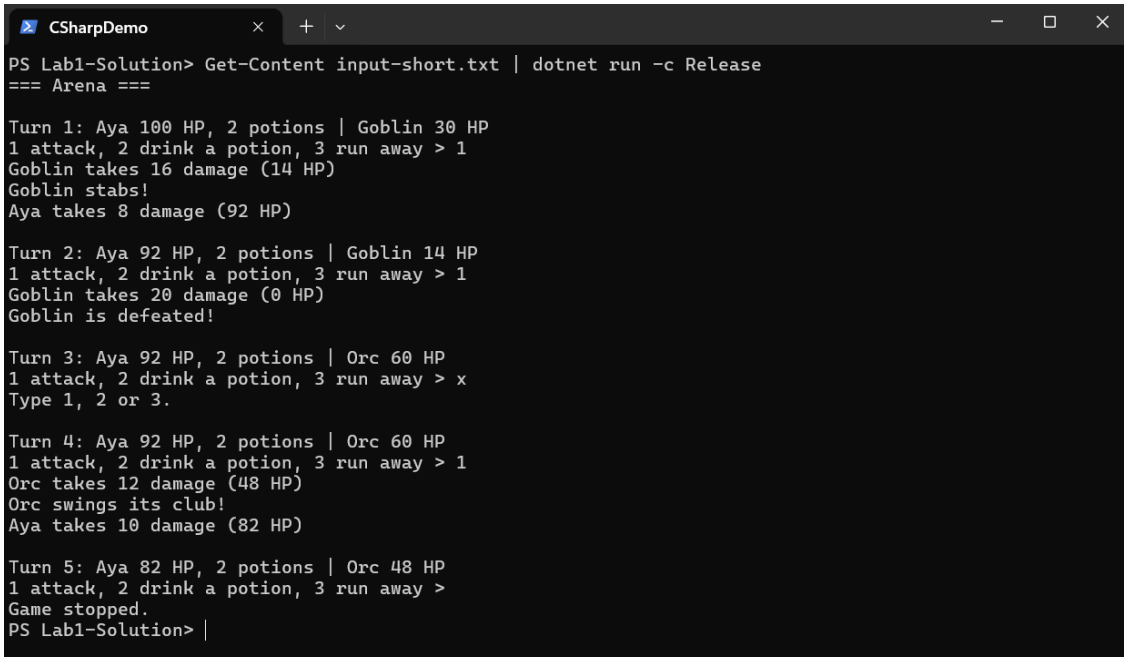


Figure 6: The finished game. Here the choices come from a file (1, 1, x, 1) so that the run can be repeated; in class, you type them.

What you are going to build

The program has three kinds of objects. `Player` keeps Aya's health between 0 and her maximum and counts her potions. `Enemy` is an *abstract* class: it describes what all enemies share (a name, a health, taking damage), but leaves each child class its own way of attacking.

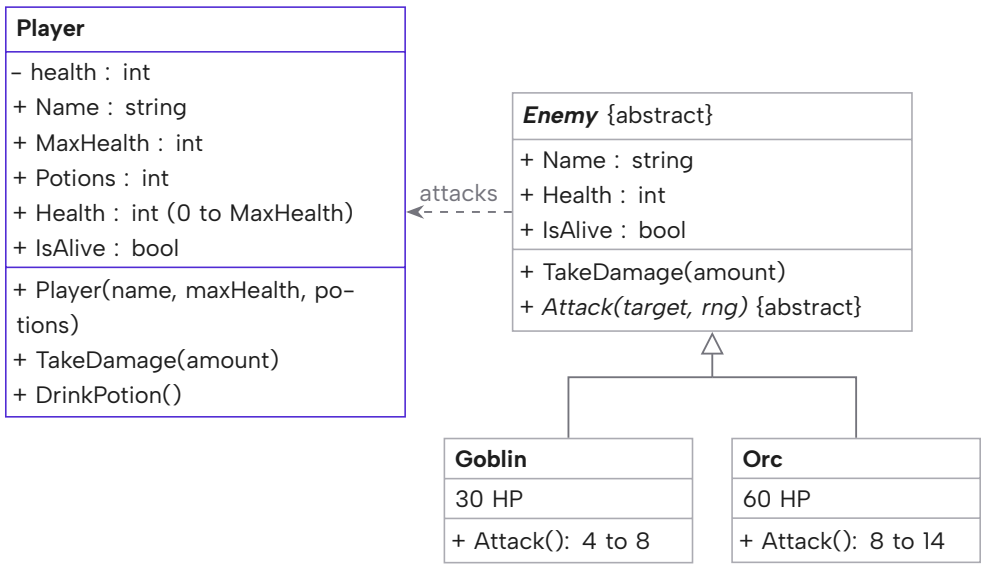


Figure 7: The classes of the duel. The numbers under Attack() are the possible damage.

The main program creates Aya and the list of enemies, then repeats turns while Aya is alive and enemies are left.

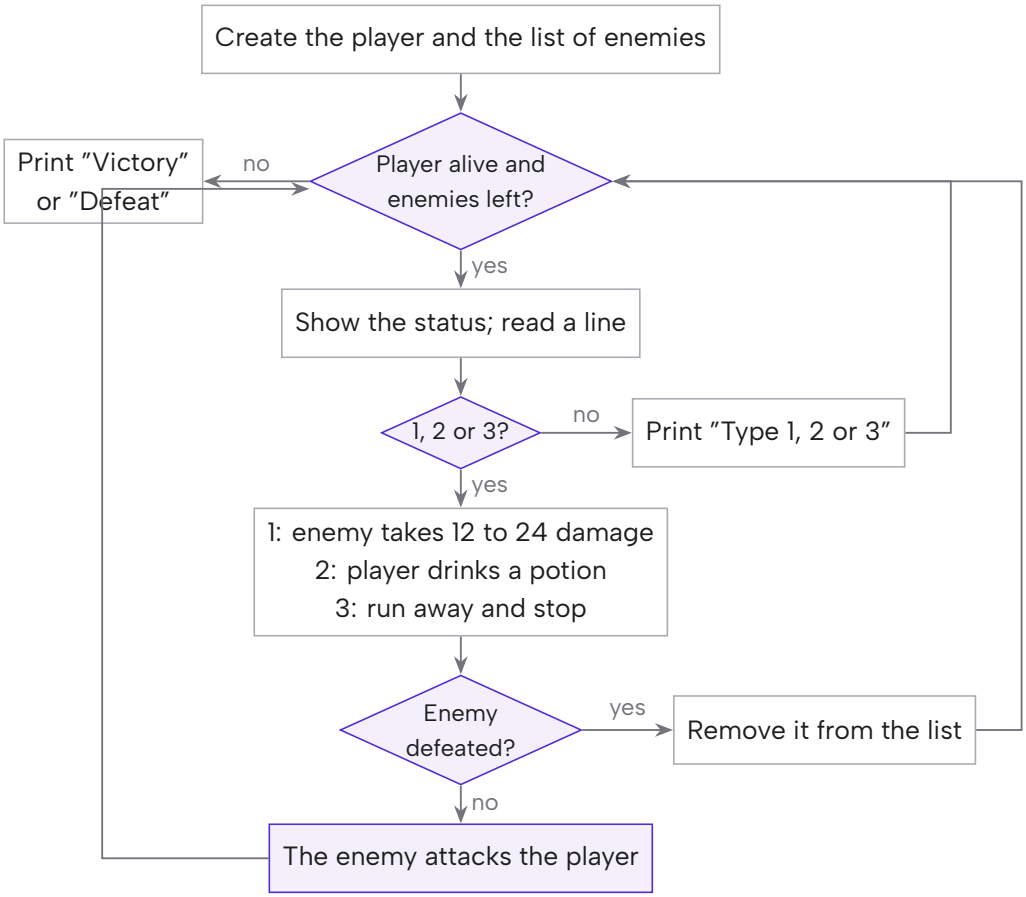


Figure 8: The game loop. Each pass through the loop is one turn. Step 5 writes the loop without the choice (Aya always attacks); step 6 adds the choice.

Table 3: The seven steps of the duel.

Step	You write	Idea	Notes
1	The `Arena` project and the starter file	Console project	1.1
2	The `Player` class	Class, private field, property, method	1.8
3	`Enemy`, `Goblin`, `Orc`	Inheritance, abstract method, `Random`	1.10
4	The wave of enemies	`List<T>`, `foreach`	1.7, 1.5
5	The game loop	`while`, `continue`, counter	1.5
6	The player's choice	`Console.ReadLine`, `int.TryParse`, `switch`	1.6, 1.4

Step	You write	Idea	Notes
7	The end of the game and the checks	Testing by hand	1.15

Step 1: the project

In your working folder: `dotnet new console -n Arena`, then `cd Arena` and `code .`. Replace the whole content of `Program.cs` with the starter file below (it is also handed out with this document). It compiles as it is; the `TODO` lines say what each step adds.

Program.cs (starter file)

```
// Lab 1 starter: a turn-based duel in the console.
// It builds and runs as it is. Fill in the TODO parts, one step at a time.
using System;
using System.Collections.Generic;

Player player = new Player("Aya", 100, 2);
List<Enemy> enemies = new List<Enemy>();
// TODO step 4: add a Goblin, an Orc and a second Goblin to the list

Console.WriteLine("=== Arena ===");
// TODO step 5: the game loop
//   while the player is alive and enemies remain:
//     show the status, read a choice, act, then the first enemy attacks
// TODO step 7: print "Victory" or "Defeat"

public class Player
{
    // TODO step 2: Name, MaxHealth, Potions, Health (kept between 0 and
    // MaxHealth), IsAlive, TakeDamage(int amount), DrinkPotion()
    public Player(string name, int maxHealth, int potions)
    {
    }
}

public abstract class Enemy
{
    // TODO step 3: Name, Health, IsAlive, TakeDamage(int amount),
    // and an abstract Attack(Player target, Random rng)
}
```

```
// TODO step 3: public class Goblin : Enemy { ... }
// TODO step 3: public class Orc : Enemy { ... }
```

In this file, the statements of the program come first and the classes after them. C# requires this order: a class placed before the first statement gives error `CS8803` (part 4).

Checkpoint: `dotnet run` prints `=== Arena ===`.



Figure 9: Checkpoint of step 1.

Step 2: the Player class

Complete the `Player` class following Figure 7. The field `health` is private: only the property `Health` touches it, and its `set` keeps it between `0` and `MaxHealth` with `Math.Clamp` (section 1.8). This way, neither a big hit nor a potion can take the health out of that range, whatever code calls `TakeDamage` or `DrinkPotion`.

`Program.cs: the Player class, to complete`

```
public class Player
{
    private int health;

    public string Name { get; }
    public int MaxHealth { get; }
    public int Potions { get; private set; }

    public int Health
    {
        get { return health; }
        private set { health = Math.Clamp(value, 0, MaxHealth); }
    }

    public bool IsAlive => health > 0;

    public Player(string name, int maxHealth, int potions)
    {
        // TODO: copy the three parameters into the properties,
        // then start with full health
```

```
    }

    public void TakeDamage(int amount)
    {
        // TODO: lower Health, then print "Aya takes 30 damage (70 HP)"
    }

    public void DrinkPotion()
    {
        // TODO: no potion left -> print "No potion left!" and stop (return)
        // otherwise: one potion less, 30 health more, print a line
    }
}
```

To test the class before writing the game, put these lines at the top of the file for now, in place of the starter statements:

Program.cs: test lines of step 2

```
Player player = new Player("Aya", 100, 2);
player.TakeDamage(30);
player.DrinkPotion();
player.DrinkPotion();
player.DrinkPotion();
player.TakeDamage(150);
Console.WriteLine($"Alive: {player.IsAlive}");
```

Checkpoint: health goes back up to 100 and no further, the third potion is missing, and 150 damage leaves health at 0, not at −80.

Output of step 2

```
Aya takes 30 damage (70 HP)
Aya drinks a potion (100 HP)
Aya drinks a potion (100 HP)
No potion left!
Aya takes 150 damage (0 HP)
Alive: False
```

Step 3: the enemies

Write the abstract class `Enemy` and its two child classes (section 1.10). The constructor of `Goblin` calls the one of `Enemy` with `base("Goblin", 30)`: the parent class stores the name and the health. Each child class writes its own version of `Attack`, marked `override`.

`Program.cs: the enemies, to complete`

```
public abstract class Enemy
{
    public string Name { get; }
    public int Health { get; private set; }
    public bool IsAlive => Health > 0;

    protected Enemy(string name, int health)
    {
        Name = name;
        Health = health;
    }

    public void TakeDamage(int amount)
    {
        // TODO: lower Health, never below 0 (Math.Max), print a line
    }

    // no body: each kind of enemy attacks in its own way
    public abstract void Attack(Player target, Random rng);
}

public class Goblin : Enemy
{
    public Goblin() : base("Goblin", 30)
    {
    }

    public override void Attack(Player target, Random rng)
    {
        Console.WriteLine("Goblin stabs!");
        target.TakeDamage(rng.Next(4, 9));     // 4 to 8: the upper bound is excluded
    }
}

// TODO: the Orc, 60 HP, "Orc swings its club!", 8 to 14 damage
```

`Random` produces the random numbers: `rng.Next(4, 9)` returns a whole number from 4 to 8, because the upper bound is excluded. Create a single `Random` object for the whole program and pass it to `Attack`: the seed is then set in one place, and `new Random(7)` is enough to replay exactly the same game while you look for a mistake.

Test with these lines, still temporary:

`Program.cs: test lines of step 3`

```
Random rng = new Random(7);          // fixed seed: the same numbers at every run
Player player = new Player("Aya", 100, 2);

Enemy goblin = new Goblin();
Enemy orc = new Orc();
goblin.Attack(player, rng);
orc.Attack(player, rng);
orc.TakeDamage(25);
Console.WriteLine($"{orc.Name} alive: {orc.IsAlive}");
orc.TakeDamage(40);
Console.WriteLine($"{orc.Name} alive: {orc.IsAlive}");
```

Checkpoint: each enemy attacks with its own message, and the Orc is no longer alive after 65 damage. With `new Random()` instead of `new Random(7)`, your damage will be different.

`Output of step 3`

```
Goblin stabs!
Aya takes 5 damage (95 HP)
Orc swings its club!
Aya takes 14 damage (81 HP)
Orc takes 25 damage (35 HP)
Orc alive: True
Orc takes 40 damage (0 HP)
Orc alive: False
```

Also try `Enemy e = new Enemy("Bat", 10);`: the compiler refuses, because an abstract class cannot be created directly (error `CS0144`). Then delete that line.

Step 4: the wave

Replace the test lines with the real start of the program: Aya, then a `List<Enemy>` filled with a Goblin, an Orc and a second Goblin (section 1.7). The list is of type `Enemy`, but it holds `Goblin` and `Orc` objects: an object of a child class fits wherever the parent class is expected.

Program.cs: start of the program at step 4

```
Player player = new Player("Aya", 100, 2);
List<Enemy> enemies = new List<Enemy>();
enemies.Add(new Goblin());
enemies.Add(new Orc());
enemies.Add(new Goblin());

Console.WriteLine("=== Arena ===");
Console.WriteLine($"{player.Name} faces {enemies.Count} enemies:");
foreach (Enemy e in enemies)
{
    Console.WriteLine($"- {e.Name}, {e.Health} HP");
}
Console.WriteLine($"First in line: {enemies[0].Name}");
```

Checkpoint:

Output of step 4

```
=== Arena ===
Aya faces 3 enemies:
- Goblin, 30 HP
- Orc, 60 HP
- Goblin, 30 HP
First in line: Goblin
```

Step 5: the game loop

Write the `while` loop of Figure 8, first without the choice: each turn, Aya attacks. The current enemy is always `enemies[0]`. When it is defeated, `enemies.RemoveAt(0)` removes it and the next one takes its place; `continue` moves to the next turn, so a defeated enemy does not strike back. The counter `turn` goes up by 1 each turn.

Program.cs: the loop, to complete

```
Random rng = new Random();
int turn = 0;

Console.WriteLine("=== Arena ===");
while (player.IsAlive && enemies.Count > 0)
{
    turn++;
```

```
    Enemy enemy = enemies[0];              // the enemy in front of Aya
    Console.WriteLine();
    Console.WriteLine($"Turn {turn}: {player.Name} {player.Health} HP | {enemy.Name}
    ↪  {enemy.Health} HP");

    // TODO: Aya attacks: the enemy takes 12 to 24 damage

    // TODO: if the enemy is defeated: print it, remove it from the list
    //       (enemies.RemoveAt(0)) and go to the next turn (continue)

    // TODO: otherwise the enemy attacks Aya
}
// TODO: print "Victory in 8 turns!" or "Defeat..."
```

Checkpoint: the fight runs by itself to the end and finishes with a victory or a defeat.

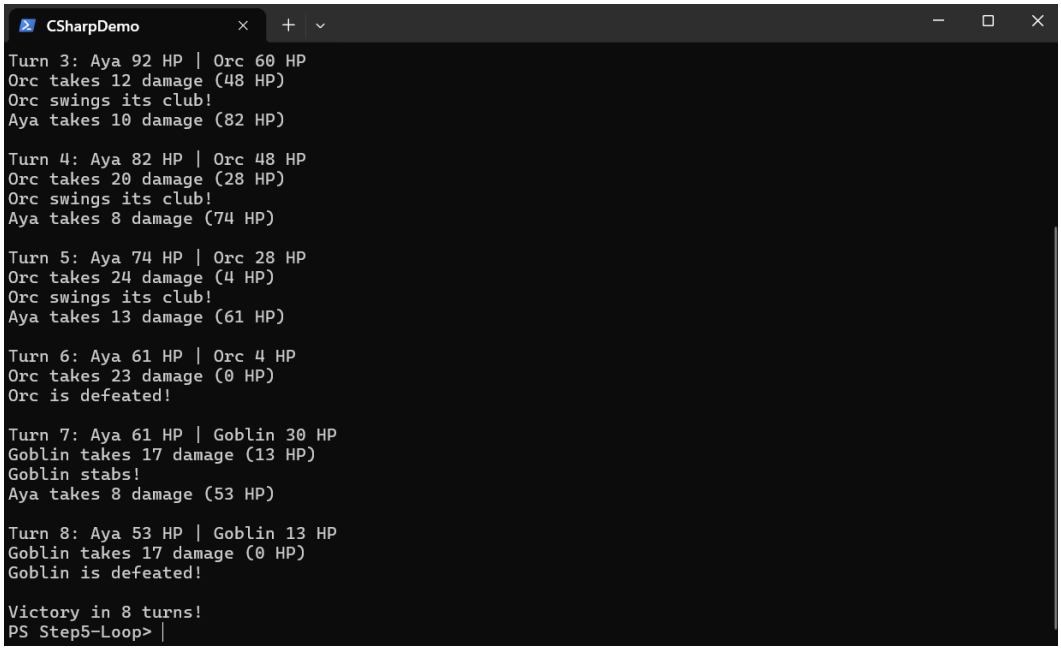


Figure 10: Checkpoint of step 5: the end of an automatic fight (seed 7).

> **Watch out**
>
> If the program never stops and fills the screen, the condition of the `while` never becomes false: a defeated enemy is not removed, or health does not go down. Stop the program with **Ctrl+C**.

Step 6: the player's choice

At the start of each turn, read the choice with `Console.ReadLine()`. This method returns text: to get a number, use `int.TryParse` (section 1.6). Unlike `int.Parse`, it does not throw an exception on x or on an empty line: it returns `false`. A `switch` then picks the action (section 1.4).

Program.cs: the choice, inside the loop, to complete

```
Console.Write("1 attack, 2 drink a potion, 3 run away > ");
string text = Console.ReadLine();
if (!int.TryParse(text, out int choice))
{
    Console.WriteLine("Type 1, 2 or 3.");
    continue;                              // same turn again
}
switch (choice)
{
    case 1:
        // TODO: the attack of step 5
        break;
    case 2:
        // TODO: Aya drinks a potion
        break;
    case 3:
        Console.WriteLine("You run away.");
        return;                            // ends the program
    default:
        // TODO: any other number: the same help line, then continue
        break;
}
```

This code replaces the automatic attack of step 5. Keep the rest of the loop: defeated enemy or counter-attack. In case 2, the potion uses Aya's turn, and the enemy still strikes back. Also switch to `new Random()`.

Checkpoint: any input that is not 1, 2 or 3 prints the help line and gives the same turn again, without a crash.

Part of a game: x, then 1

```
Turn 3: Aya 92 HP, 2 potions | Orc 60 HP
1 attack, 2 drink a potion, 3 run away > x
Type 1, 2 or 3.
```

```
Turn 4: Aya 92 HP, 2 potions | Orc 60 HP
1 attack, 2 drink a potion, 3 run away > 1
Orc takes 12 damage (48 HP)
```

Step 7: the end of the game

After the loop, print `Victory in 10 turns!` (with the real number of turns) or `Defeat...`. Then play several games and check each case by hand:

- **Victory**: attacking every turn, the three enemies fall.
- **Defeat**: drinking potions without attacking, Aya ends up losing; health stops at `0`.
- **Potions**: the third potion prints `No potion left!`.
- **Input**: x, `7` and an empty line print the help line.
- **Running away**: `3` prints `You run away.` and stops the program.

Checkpoint: a full game like this one.

End of a full game

```
Turn 7: Aya 61 HP, 2 potions | Orc 4 HP
1 attack, 2 drink a potion, 3 run away > 2
Aya drinks a potion (91 HP)
Orc swings its club!
Aya takes 13 damage (78 HP)

Turn 8: Aya 78 HP, 1 potions | Orc 4 HP
1 attack, 2 drink a potion, 3 run away > 1
Orc takes 17 damage (0 HP)
Orc is defeated!

Turn 9: Aya 78 HP, 1 potions | Goblin 30 HP
1 attack, 2 drink a potion, 3 run away > 1
Goblin takes 24 damage (6 HP)
Goblin stabs!
Aya takes 6 damage (72 HP)

Turn 10: Aya 72 HP, 1 potions | Goblin 6 HP
1 attack, 2 drink a potion, 3 run away > 1
Goblin takes 13 damage (0 HP)
Goblin is defeated!

Victory in 10 turns!
```

If you finish early, go on to part 3.

3 Going further

Four exercises, to do at home, in order. Each one adds to the duel an idea from the end of Part 1 of the course, and each one carries over to Unity as it is. Start from your program of part 2. The outputs below come from a version that does all four exercises; in that version, choice 3 (running away) is gone, because exercise 3 offers to play again.

Table 4: The four exercises.

Exercise	You add	Idea	Notes	In Unity
1	A boss with armour	`virtual`, `override`, `base`	1.10	An enemy that inherits from a shared script
2	A `Died` event	`event`, subscribing with `+=`, lambda	1.13	`ScoreCounter` subscribes to `Collected`
3	Several games and the best ones	Method that returns a value, LINQ	1.6, 1.14	Sorting scores, finding objects
4	A results file	`File.AppendAllText`, `try`/`catch`	1.12	Saving a game

Exercise 1: a boss with armour

Add a `Boss` at the end of the wave: 60 health points and an armour of 6, which removes 6 points from every hit it takes (at least 1 damage always gets through). The boss must therefore *change* the way an enemy takes damage.

1. In `Enemy`, mark `TakeDamage` as `virtual`: a child class is now allowed to replace it.
2. In `Boss`, write `public override void TakeDamage(int amount)`. It reduces the damage, then calls `base.TakeDamage(reduced)`: the version in `Enemy` does the rest (lower the health, print the line).
3. Add `new Boss()` to the list.

`Program.cs: the boss, to complete`

```
// in Enemy: virtual allows a child class to replace the method
public virtual void TakeDamage(int amount) { /* as before */ }

public class Boss : Enemy
{
    public int Armour { get; } = 6;

    public Boss() : base("Boss", 60)
    {
    }

    public override void TakeDamage(int amount)
    {
        int reduced = Math.Max(amount - Armour, 1);
        Console.WriteLine($"The armour absorbs {amount - reduced} damage.");
        base.TakeDamage(reduced);          // the version of Enemy does the rest
    }

    // TODO: Attack, "Boss slams the ground!", 9 to 16 damage
}
```

Checkpoint: each hit on the boss prints what the armour absorbs.

`One turn against the boss: 17 damage, 6 of them absorbed`

```
Turn 7: Aya 61 HP, 2 potions | Boss 60 HP
1 attack, 2 drink a potion > 1
The armour absorbs 6 damage.
Boss takes 11 damage (49 HP)
Boss slams the ground!
Aya takes 16 damage (45 HP)
```

Question. Without the word `virtual` in `Enemy`, what happens? Try it, read the error, then put it back.

Exercise 2: a Died event

For now, the game loop checks whether Aya is alive. With an *event*, `Player` announces its own death, and any other part of the program can listen, without `Player` knowing who listens (section 1.13). This is how Unity scripts talk to each other without knowing each other.

`Program.cs: the Died event`

```
// in Player
public event Action<string> Died;

public void TakeDamage(int amount)
{
    bool wasAlive = IsAlive;
    Health -= amount;
    Console.WriteLine($"{Name} takes {amount} damage ({Health} HP)");
    if (wasAlive && !IsAlive)
    {
        Died?.Invoke(Name);               // ?. : only if someone subscribed
    }
}

// in the game, after creating the player
player.Died += name => Console.WriteLine($"Game over for {name}.");
```

`Action<string>` is the type of a method that receives a `string` and returns nothing. `Died?.Invoke(Name)` calls every subscriber, and the `?.` avoids an error when nobody has subscribed. The variable `wasAlive` makes sure the event fires only once, at the exact moment health reaches `0`.

Checkpoint: the line `Game over for Aya.` appears right after the final blow.

The last turn of a defeat

```
Turn 10: Aya 9 HP, 2 potions | Boss 24 HP
1 attack, 2 drink a potion > 1
The armour absorbs 6 damage.
Boss takes 15 damage (9 HP)
Boss slams the ground!
Aya takes 10 damage (0 HP)
Game over for Aya.
```

Exercise 3: several games and the best ones

Move the game loop into a method `int PlayGame()`, which creates Aya and the enemies, plays one game and returns the number of turns of a victory, or `0` for a defeat. The main program plays games while the answer to `Play again? (y/n)` is `y`, and keeps the turns of the victories in a `List<int>`. At the end, LINQ sorts the list and keeps the three best: `OrderBy(t => t)` sorts from smallest to largest, `Take(3)` keeps the first three (section 1.14).

Program.cs: several games

```
using System.Linq;

List<int> wins = new List<int>();          // turns of each won game
int game = 0;
while (true)
{
    game++;
    int turns = PlayGame();                // your game loop, moved into a method:
    if (turns > 0)                         // it returns the turns of a victory, or 0
    {
        wins.Add(turns);
    }
    Console.Write("Play again? (y/n) > ");
    if (Console.ReadLine() != "y")
    {
        break;
    }
}
Console.WriteLine($"Victories: {wins.Count} of {game}");
foreach (int t in wins.OrderBy(t => t).Take(3))
{
    Console.WriteLine($"- {t} turns");
}
```

Checkpoint: after the last game, the summary and the best victories.

```
End of the third game and summary
```

```
Saved: Game 3: defeat
Play again? (y/n) > n

Victories: 2 of 3
- 11 turns
- 13 turns
```

Exercise 4: a results file

After each game, add a line to the file `results.txt`. `File.AppendAllText` creates the file if it does not exist and writes at the end otherwise. Writing to a file can fail for reasons the program does not control (file locked by another program, full disk, read-only file): the call goes inside a `try`, and a `catch` prints a message instead of letting the program crash (section 1.12). Two are needed here, because these

problems come as two types of exception: `IOException` for a locked file or a full disk, `UnauthorizedAccessException` for a file or folder you are not allowed to write to.

`Program.cs: saving a result`

```
using System.IO;

void SaveResult(string line)
{
    try
    {
        File.AppendAllText("results.txt", line + Environment.NewLine);
        Console.WriteLine($"Saved: {line}");
    }
    catch (IOException e)                // file locked by another program, disk full...
    {
        Console.WriteLine($"Could not save the result: {e.Message}");
    }
    catch (UnauthorizedAccessException e) // read-only file, protected folder
    {
        Console.WriteLine($"Could not save the result: {e.Message}");
    }
}
```

Checkpoint: the file `results.txt` appears in the project folder, one line per game.

`results.txt after three games`

```
Game 1: victory in 11 turns
Game 2: victory in 13 turns
Game 3: defeat
```

To see a `catch` at work, make `results.txt` read-only (right-click, **Properties**, tick **Read-only**), then play a game: the program prints `Could not save the result: Access to the path '...' is denied.` and goes on. Untick the box afterwards.

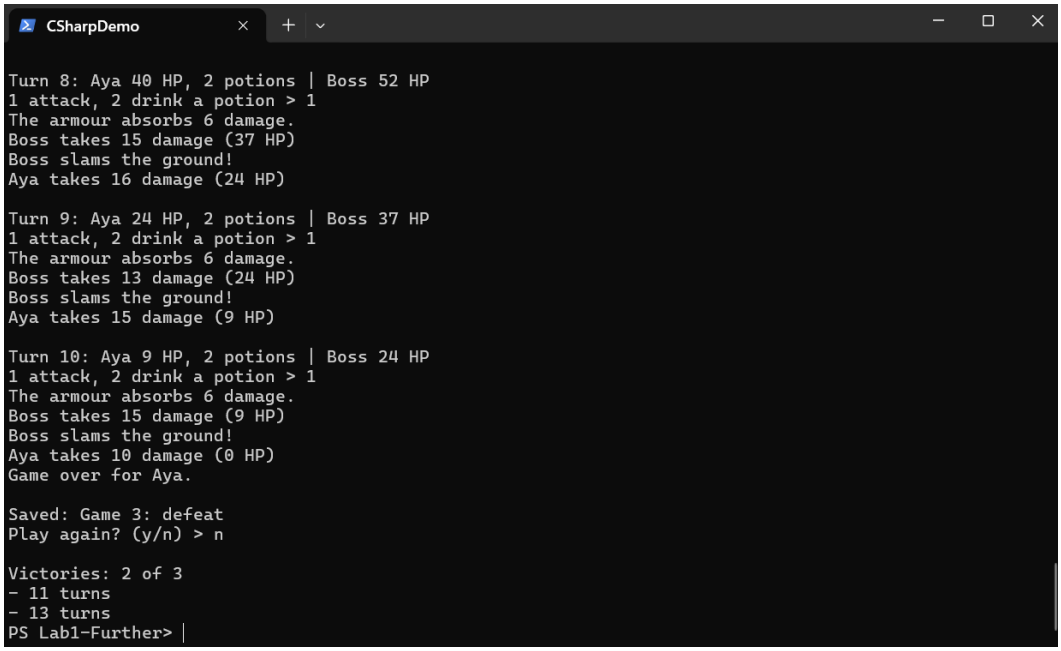


Figure 11: The four exercises together: the end of a defeat, the saving and the summary.

4 Reference

Commands

Table 5: The commands of the lab.

Command	Effect
`dotnet --version`	Prints the version of the installed SDK
`dotnet new console -n Arena`	Creates the folder `Arena` with a console project
`dotnet run`	Compiles the project of the current folder, then runs it
`dotnet build`	Only compiles, and prints the errors
`code .`	Opens the current folder in VS Code
Ctrl+C	Stops the program running in the terminal
Up arrow	Brings back the previous command

Error messages

The messages below are copied from real builds. The position `(line,column)` changes with your file. Section 1.15 of the notes explains how to read a message and gives more of them.

Table 6: Frequent compiler errors in this lab.

Code	Message	Likely cause	Fix
CS1002	`; expected`	A `;` is missing at the end of a statement	Add the `;` at the position given
CS1003	`Syntax error, ',' expected`	A `;` is missing after a declaration	Look at the end of the line given
CS1513	`} expected`	A closing brace is missing	Count the `{` and `}` of the block
CS0103	`The name 'console' does not exist in the current context`	A typo or a wrong capital, or a variable declared elsewhere	`Console`, with a capital; declare the variable first
CS0246	`The type or namespace name 'List<>' could not be found (are you missing a using directive or an assembly reference?)`	A `using` is missing	`using System.Collections.Generic;`
CS0029	`Cannot implicitly convert type 'string' to 'int'`	Text stored in a number	`int.TryParse` (step 6)
CS0165	`Use of unassigned local variable 'lives'`	A variable read before it got a value	Give it a value where it is declared
CS0122	`'Player.health' is inaccessible due to its protection level`	A `private` member used outside its class	Go through the `Health` property
CS0144	`Cannot create an instance of the abstract type or interface 'Enemy'`	`new` on an abstract class	Create a `Goblin` or an `Orc`

Code	Message	Likely cause	Fix
CS0534	`'Orc' does not implement inherited abstract member 'Enemy.Attack()'`	The child class did not write the abstract method	Write `public override void Attack(...)`
CS0506	`'Boss.TakeDamage(int)': cannot override inherited member 'Enemy.TakeDamage(int)' because it is not marked virtual, abstract, or override`	`override` without `virtual` in the parent class	Add `virtual` in `Enemy`
CS0161	`'Status(int)': not all code paths return a value`	One path through the method ends without `return`	Add a `return` or a final `else`
CS8803	`Top-level statements must precede namespace and type declarations.`	A class placed before the statements	Put the classes at the end of the file

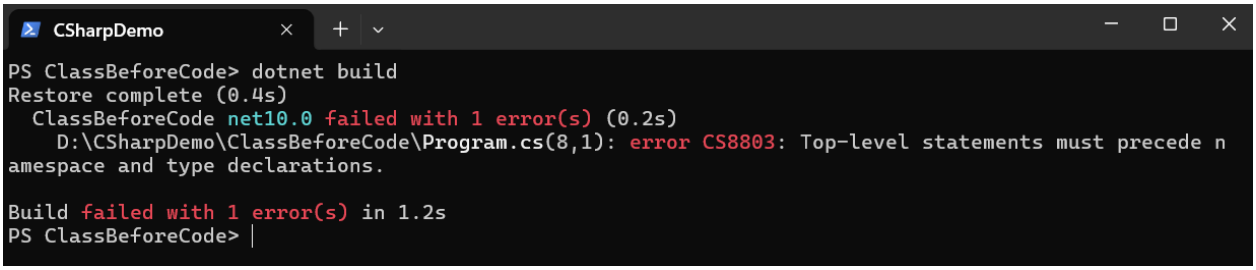


Figure 12: Error CS8803: the class `Player` is written before the first statement of the program.

Two errors only show up when the program runs. It compiles, then stops with `Unhandled exception`:

Table 7: Frequent runtime errors in this lab.

Exception	Message	Likely cause
`ArgumentOutOfRangeException`	`Index was out of range. Must be non-negative and less than the size of the collection. (Parameter 'index')`	`enemies[0]` on an empty list: the loop does not check `enemies.Count > 0`
`FormatException`	`The input string 'x' was not in a correct format.`	`int.Parse` on input that is not a number: use `int.TryParse`

Frequent questions

The terminal answers that `dotnet` is not recognized. The SDK is not installed, or the terminal was opened before the installation. Close every terminal (and VS Code), open a new one, then type `dotnet --version` again. If the problem remains, install the .NET 10 SDK again (notes, section 0.8).

The program does not read what I type. Run it with `dotnet run` in the terminal. With **F5** in VS Code, depending on the configuration, the program can run in the *Debug Console*, which does not accept input.

The program never stops. The condition of a loop never becomes false. **Ctrl+C** stops it; then look for what should change at each turn (step 5).

My numbers are not the ones in this document. That is expected with `new Random()`. To get exactly the same ones, use `new Random(7)`.

A comma appears instead of the dot (`5,5`). The display of numbers follows the language of Windows. The code itself is always written `5.5f`.

VS Code underlines code in red, but `dotnet run` works. The extension has not finished loading the project. Wait a few seconds, or close and reopen the folder. `dotnet build` is the reference.

`Program.cs` has errors I do not understand after a copy and paste. Check that there is only one block of statements at the top of the file, and that each class appears only once.

From the lab to the lecture notes

Table 8: Where to find each idea in the lecture notes.

Idea	Notes	Idea	Notes
First program, `dotnet`	1.1	Classes, properties	1.8
Variables and types	1.2	Inheritance, `abstract`, `virtual`	1.10
Operators	1.3	Exceptions, `try`/`catch`	1.12
`if`, `switch`	1.4	Delegates and events	1.13
Loops	1.5	LINQ	1.14
Methods, `int.TryParse`	1.6	Frequent errors	1.15
`List<T>`	1.7	From C# to Unity	1.16

Handing in. Zip the `Arena` folder without its `bin` and `obj` subfolders: `dotnet build` recreates them, and they are much larger than the code.